

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнил: М. А. Хорохорин
Группа: М8О-306БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: познакомиться с основами 3D-графики — построением простых 3D-объектов, проекцией на 2D-плоскость, матрицами перспективной и ортографической проекции и аффинными преобразованиями. Программа должна работать в реальном времени на Vulkan, GLFW и ImGui и позволять менять проекцию и преобразования объектов.

Вариант 4: усечённый правильный тетраэдр, грани которого — 4 правильных треугольника и 4 правильных шестиугольника.

Выполнены все дополнительные задания: переключение ортографической и перспективной проекции; изменение позиции, поворота и масштаба фигуры; движение и вращение по траектории с паузой, воспроизведением и настройкой скорости и параметров траектории; выбор цвета фигуры; процедурные цвета вершин, умножаемые на выбранный цвет; отрисовка нескольких объектов с отдельным набором дескрипторов у каждого.

Метод решения

Геометрия. Берётся правильный тетраэдр с вершинами $(1, 1, 1)$, $(1, -1, -1)$, $(-1, 1, -1)$, $(-1, -1, 1)$. Каждое ребро $A_i A_j$ делится на три части, точка у вершины A_i равна $P_{ij} = (2A_i + A_j) / 3$. Получается 12 вершин, 18 рёбер и 8 граней ($V - E + F = 2$). У каждой вершины тетраэдра срезается треугольник, каждая исходная грань становится шестиугольником. Вершины нормализуются, поэтому фигура вписана в единичную сферу. Шестиугольники разбиваются веером на 4 треугольника, всего 20 треугольников и 60 индексов. Обход каждой грани выбирается так, чтобы нормаль $(b - a) \times (c - a)$ смотрела наружу.

Цвет вершины вычисляется из её локальной позиции: $\text{color} = \text{position} \cdot 0.5 + 0.5$. В вершинном шейдере он умножается на цвет, выбранный в интерфейсе; при выключенных цветах вершин остаётся только выбранный цвет. Чтобы грани одного цвета не сливались, во фрагментном шейдере добавлено плоское затенение: нормаль грани равна векторному произведению производных позиции в системе камеры по экранным осям ($dFdx \times dFdy$), яркость считается по закону Ламберта: $0.35 + 0.65 \cdot \max(n \cdot l, 0)$.

Преобразования. Матрица модели $M = T \cdot R_y \cdot R_x \cdot R_z \cdot S$: сначала масштаб, затем повороты, затем перенос. Камера вращается вокруг начала координат, матрица вида строится функцией `lookAt`. Матрицы проекции

записаны вручную под систему координат Vulkan, где ось Y направлена вниз, а глубина лежит в $[0, 1]$. Для перспективы с $f = 1 / \tan(\text{fov} / 2)$: $x' = f \cdot x / a$, $y' = -f \cdot y$, $z' = \text{far} \cdot z / (\text{near} - \text{far}) + \text{near} \cdot \text{far} / (\text{near} - \text{far})$, $w' = -z$, поэтому ближняя плоскость переходит в глубину 0, дальняя — в 1. Ортогографическая проекция: $x' = x / (h \cdot a)$, $y' = -y / h$, $z' = (z + \text{near}) / (\text{near} - \text{far})$, $w' = 1$, размер не зависит от расстояния.

Анимация. Объект движется по эпитрохоиде с вертикальной волной: $x = R \cdot \cos t + r \cdot \cos kt$, $y = H \cdot \sin 2t$, $z = R \cdot \sin t + r \cdot \sin kt$, и одновременно вращается со скоростью spin . Время анимации накапливается как $t += dt \cdot \text{speed}$ только в режиме воспроизведения, поэтому пауза и смена скорости не дают рывков.

Объекты Vulkan. Вершинный и индексный буферы (`VkBuffer`) выделяются через Vulkan Memory Allocator в памяти, видимой процессору, и заполняются через `memcpu`. Для каждого кадра используются `uniform`-буферы: один для камеры (`view`, `projection`) и по одному на объект (`model`, цвет, веса цветов вершин и затенения). Раскладка C++-структур совпадает с `std140`, что проверяется `static_assert`. Созданы два `VkDescriptorSetLayout`, `VkDescriptorPool` на 9 наборов, набор 0 для камеры и 8 наборов 1 для объектов, `VkPipelineLayout` с двумя наборами и `VkPipeline`: вершины из одного буфера с позицией и цветом, список треугольников, отсечение задних граней, тест и запись глубины, динамические `viewport` и `scissor`. Шейдеры на GLSL компилируются в SPIR-V утилитой `glslc`. В основной `VkRenderPass` стартового кода добавлена внешняя зависимость подпрохода (`VkSubpassDependency`), которая упорядочивает переход `layout` изображения `swarchain` после его получения; без неё синхронизационная валидация сообщала об ошибке `WRITE_AFTER_READ`.

Кадр. В `update` рисуется интерфейс `ImGui` и на процессоре считаются матрицы. В `render`, после ожидания `fence` предыдущего кадра, данные копируются в `uniform`-буферы и записывается командный буфер: начало `render pass` с очисткой цвета и глубины, установка `viewport` и `scissor`, привязка конвейера, буферов и набора камеры, затем для каждого объекта привязка его набора и `vkCmdDrawIndexed`.

Геометрия и матрицы вынесены в отдельную библиотеку без зависимости от Vulkan и покрыты модульными тестами `GoogleTest`. Программа проверена со слоями валидации Vulkan, включая синхронизационную валидацию и `best practices`: сообщений нет.

Результаты

На рисунках показана работа программы: слева окно настроек сцены (проекция, камера, список объектов), справа окно выбранного объекта (преобразования, цвет, анимация), в центре сцена.

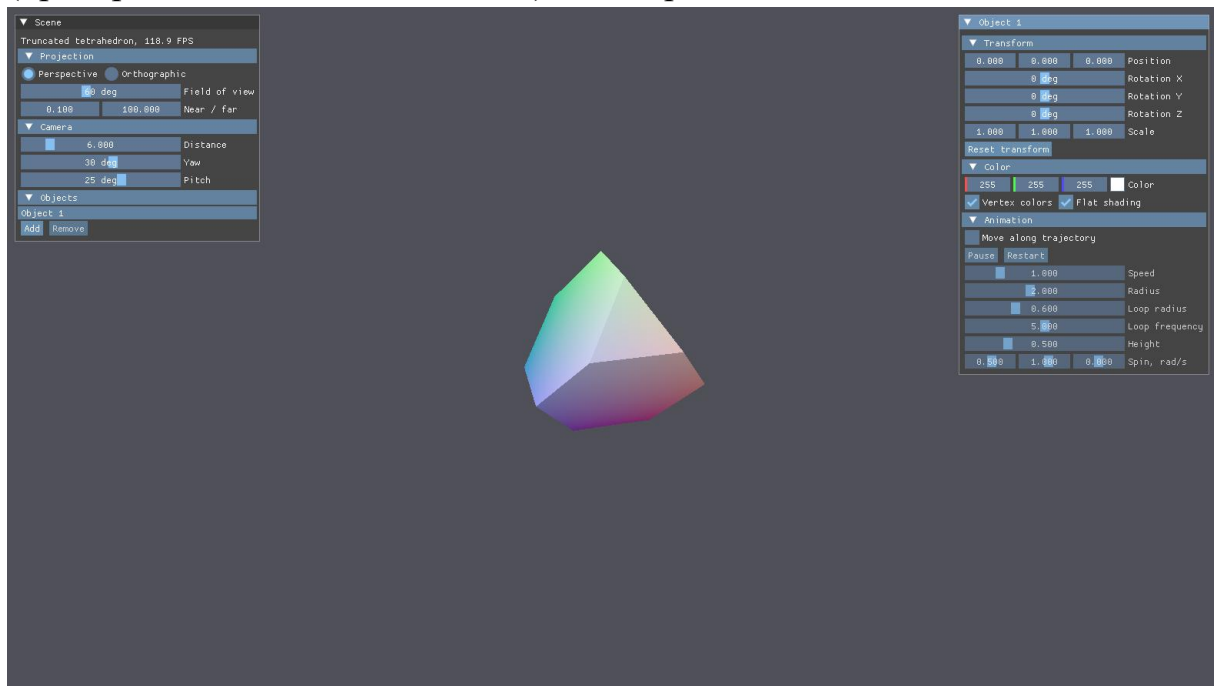


Рисунок 1 — перспективная проекция, цвета вершин включены

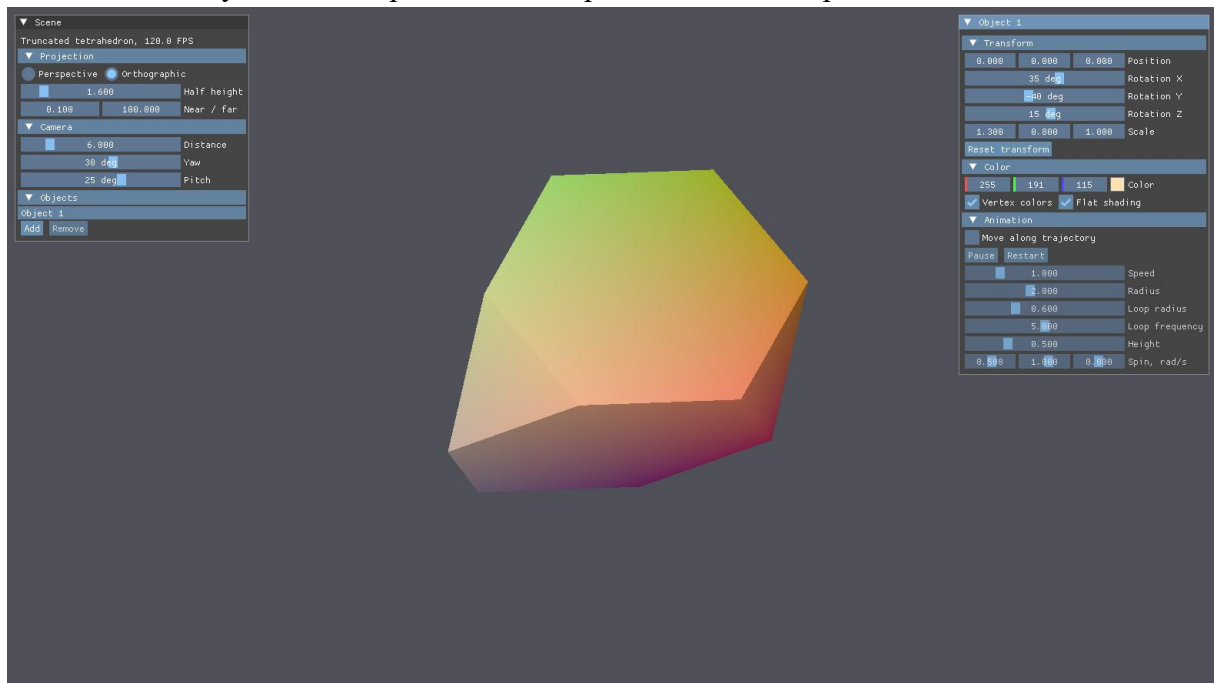


Рисунок 2 — ортографическая проекция, изменены поворот, масштаб и цвет

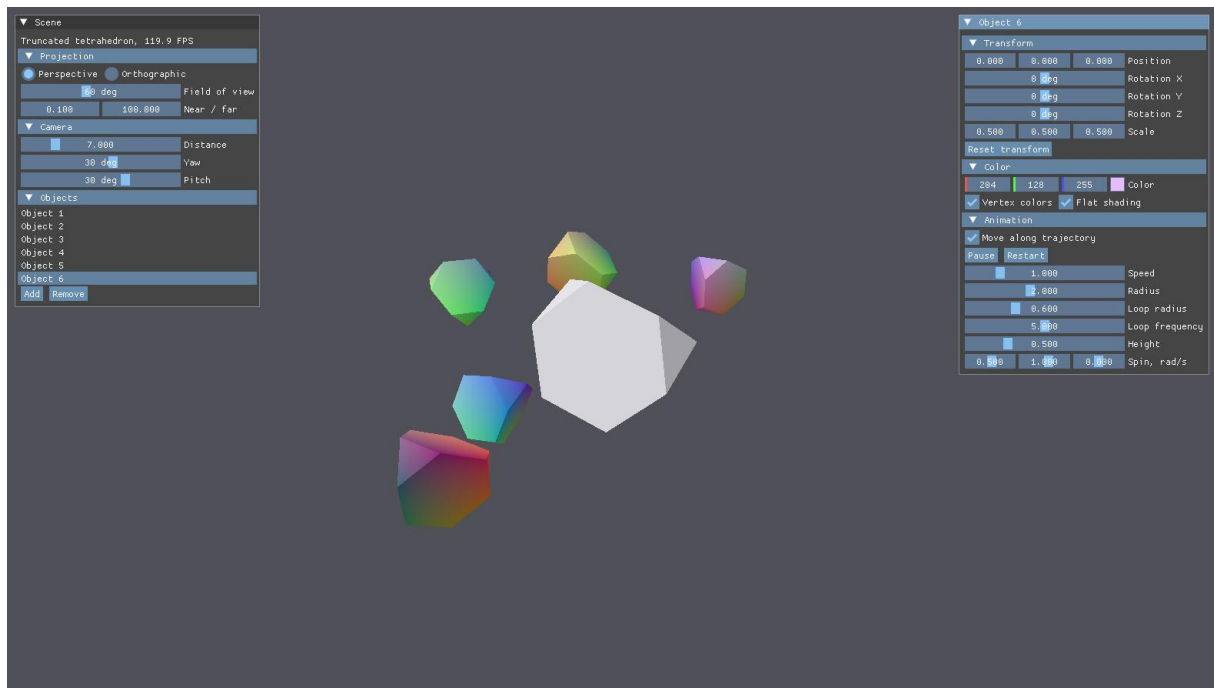


Рисунок 3 — несколько объектов движутся по траектории

Выводы

Я научился строить трёхмерную фигуру из вершин и индексов, передавать данные в шейдеры через вершинные и uniform-буферы, описывать графический конвейер Vulkan и наборы дескрипторов, выводить матрицы перспективной и ортографической проекции для системы координат Vulkan и составлять аффинные преобразования. Также разобрался, почему важен порядок обхода треугольников при отсечении задних граней и как выравнивать данные по правилам std140.